

Universidad Veracruzana

LICENCIATURA EN INGENIERÍA DE SOFTWARE

Principios de Diseño de Software

**Refactorización Arquitectónica:
Proyecto Superhéroes**

Veronica Hernandez Francisco

Mayo de 2026

Índice

1. Introducción	2
2. Contexto del Proyecto y Reglas de Negocio	2
3. Análisis de la Arquitectura Inicial	2
4. Implementación de Arquitectura Limpia	3
4.1. El antes y el después: <code>main.dart</code>	3
5. Aplicación de los Principios SOLID	3
5.1. S – Single Responsibility Principle (SRP)	4
5.2. O – Open/Closed Principle (OCP)	4
5.3. D – Dependency Inversion Principle (DIP)	4
6. Resumen de Patrones Implementados	4
7. Patrones Creacionales	5
7.1. 1. Singleton	5
7.2. 2. Abstract Factory	5
7.3. 3. Factory Method	5
7.4. 4. Builder	6
8. Patrones Estructurales	6
8.1. 5. Facade (Fachada)	6
8.2. 6. Adapter (Adaptador)	7
8.3. 7. Decorator (Decorador)	7
8.4. 8. Proxy	8
9. Retos Técnicos Superados	8
10. Beneficios Obtenidos y Visión a Futuro	8
11. Conclusión	9

1. Introducción

Este documento habla de la refactorización del FrontEnd del proyecto “Superheroes”. Al principio, el código funcionaba, pero era como una torre de Jenga a punto de caerse. Tenía todo amontonado, especialmente en el archivo `main.dart`, donde mezclaba el diseño de las pantallas con las peticiones a la API y el manejo de los datos crudos.

Como estudiante de ingeniería de software que actualmente cursa la materia de Principios de Diseño de Software, me di cuenta de que mantener ese código a largo plazo iba a ser imposible. Cualquier cosa que moviera, terminaría rompiendo otra parte. Estaba violando casi todos los principios de un buen diseño. Por eso, y como parte de la asignación de la materia, decidí meterle mano al proyecto y reestructurarlo por completo usando una Arquitectura Limpia (Clean Architecture).

A lo largo de este reporte, explico cómo se aplicaron diferentes Patrones de Diseño (Creacionales y Estructurales) para separar las responsabilidades de mi proyecto, logrando un código profesional, escalable y libre de errores fantasma.

2. Contexto del Proyecto y Reglas de Negocio

Para entender por qué se aplicaron ciertas soluciones arquitectónicas, primero es importante entender qué hace la aplicación. El sistema se compone de una API en Spring Boot conectada a una base de datos, y un FrontEnd móvil desarrollado en Flutter.

La aplicación permite a los usuarios visualizar un catálogo de superhéroes, leer sus habilidades y debilidades. Para interactuar, el sistema maneja las siguientes reglas de negocio clave:

- Un usuario debe estar autenticado (vía token JWT) para interactuar.
- Un usuario puede dejar comentarios en el perfil de un superhéroe.
- Un usuario puede reaccionar a un superhéroe usando diferentes emojis (Like, Corazón, Odio, etc.).
- Un usuario puede eliminar su propia reacción haciendo clic nuevamente sobre ella.

El manejo de estos flujos requería llamar a múltiples servicios web al mismo tiempo, lo que generaba un estrés enorme en el código visual de la aplicación.

3. Análisis de la Arquitectura Inicial

Antes de empezar con los cambios, tengo que admitir que mi proyecto era un desastre tecnológicamente hablando. Tenía casi todo metido en el `main.dart`: las llamadas a la API, los modelos, y hasta el diseño de las tarjetas de los superhéroes. Si quería cambiar un color, tenía que andar buscando entre cientos de líneas de código donde también estaba la lógica del login.

El problema real era que, si algo fallaba (como el error que tuve donde los comentarios no mostraban el nombre del usuario), era tardadísimo encontrar el origen del fallo porque todo estaba amarrado entre sí. El código estaba muy “acoplado” en un clásico código espagueti. Me di cuenta de que necesitaba orden y separar las cosas por carpetas, así que decidí aplicar una Arquitectura Limpia.

4. Implementación de Arquitectura Limpia

Para dar soporte a los patrones de diseño, primero tuve que organizar las carpetas del proyecto. Adopté un enfoque de *Clean Architecture*, dividiendo el proyecto en capas que interactúan de afuera hacia adentro.

Para ilustrar este gran cambio, a continuación presento el árbol de directorios final de mi carpeta `lib/`, el cual demuestra la separación física de responsabilidades:

```
lib/
|-- core/
|   |-- adapters/          # Adaptadores de JSON y API
|   |-- constants/        # Colores y URLs
|   |-- decorators/       # Envoltorios (Logs y Caché)
|   |-- facades/          # Coordinadores de la app
|   |-- factories/        # Creadores de objetos y servicios
|   |-- proxies/          # Validadores previos a la API
|
|-- data/
|   |-- services/          # Implementaciones reales (HTTP GET/POST)
|
|-- domain/
|   |-- models/            # Clases puras (Superheroe, Comment)
|   |-- repositories/     # Contratos de repositorios
|   |-- services/         # Interfaces (IAuthService)
|
|-- presentation/
|   |-- models/            # Datos específicos de la UI
|   |-- screens/           # Pantallas completas (Home, Login)
|   |-- theme/             # Tipografías y estilos
|   |-- widgets/          # Tarjetas y modales (BottomSheet)
|
|-- main.dart              # Archivo de arranque limpio
```

4.1. El antes y el después: `main.dart`

Un claro ejemplo del beneficio de esta estructura es mi archivo principal. Antes de refactorizar, mi `main.dart` superaba fácilmente las **500 líneas de código**, ya que contenía las variables de estado, las funciones de recarga, los modales y las peticiones de red.

Hoy, gracias a la delegación de responsabilidades, mi `main.dart` quedó reducido drásticamente, encargándose única y exclusivamente de arrancar la aplicación, configurar las rutas y lanzar el tema visual. Todo el trabajo pesado fue delegado a la capa **Presentation** y al **Facade**.

5. Aplicación de los Principios SOLID

Para que el código dejara de ser un espagueti, me guíé por los principios SOLID que vimos en clase. No los usé solo por compromiso, sino porque realmente ayudan a que uno no se vuelva loco cuando el proyecto crece.

5.1. S – Single Responsibility Principle (SRP)

Este principio fue el que más me sirvió. Básicamente dice que cada clase debe hacer una sola cosa. Antes, mi pantalla de inicio cargaba los datos, los dibujaba y manejaba los errores. Ahora, separé todo:

- Los modelos se encargan solo de los datos (como en `domain/models/comment.dart`).
- Los servicios solo hablan con la API (como `data/services/comment_service_impl.dart`).
- Los widgets solo se encargan de verse bonitos en pantalla.

5.2. O – Open/Closed Principle (OCP)

Aquí la idea es que el código debe estar abierto para extenderse, pero cerrado para modificarse. Lo apliqué usando interfaces. Por ejemplo, creé `IAuthService`. Si el día de mañana quiero que la app use autenticación biométrica o Firebase, no tengo que cambiar el código de las pantallas; solo creo una clase nueva que implemente esa interfaz y listo.

5.3. D – Dependency Inversion Principle (DIP)

En lugar de que mi pantalla llame directamente a una clase concreta, ahora depende de abstracciones. Mi app le pide al `ServiceFactory` que le dé un servicio. La pantalla no sabe cómo funciona el servicio por dentro. Esto me salvó la vida cuando tuve que arreglar el error del `List<dynamic>` en Chrome, porque pude cambiar la forma en que se procesaban las listas en la capa de datos sin tocar el diseño de la app.

```
// En domain/services/i_comment_service.dart
abstract class ICommentService {
  Future<List<Comment>> getCommentsBySuperheroe(int superheroeId);
  Future<void> createComment(int superheroeId, String content);
}
```

Listing 1: Ejemplo de Interfaz para cumplir con SOLID

6. Resumen de Patrones Implementados

A continuación, presento una tabla formal que resume los 8 patrones de diseño integrados en la arquitectura, clasificándolos por su tipo y el archivo donde habitan actualmente.

Tipo	Patrón	Ubicación en el Proyecto
Creacional	Singleton	<code>core/factories/service_factory.dart</code>
Creacional	Abstract Factory	<code>core/factories/service_factory.dart</code>
Creacional	Factory Method	<code>core/factories/superheroe_factory.dart</code>
Creacional	Builder	<code>core/factories/query_builder.dart</code>
Estructural	Facade	<code>core/facades/app_coordinator.dart</code>
Estructural	Adapter	<code>core/adapters/api_response_adapter.dart</code>
Estructural	Decorator (Logging)	<code>core/decorators/logging_decorator.dart</code>
Estructural	Decorator (Caching)	<code>core/decorators/cache_decorator.dart</code>
Estructural	Proxy	<code>core/proxies/service_proxy.dart</code>

Cuadro 1: Patrones de Diseño y su ubicación en el proyecto

7. Patrones Creacionales

Estos patrones me sirvieron muchísimo para controlar cómo “nacen” los objetos en mi aplicación. Antes, mi código creaba copias de servicios por todos lados, consumiendo memoria a lo tonto.

7.1. 1. Singleton

El Singleton asegura que exista una sola instancia de una clase en todo el ciclo de vida de la aplicación. Yo lo ocupé principalmente en mis fábricas. Por ejemplo, en mi archivo `service_factory.dart`, no tiene sentido crear múltiples fábricas de servicios, ¡solo necesitamos una que dirija todo!

Oculté el constructor original e hice uno interno (`_internal`) para que nadie más pueda crear copias accidentales.

```
class ServiceFactory {
  // Solo existe esta instancia en toda la app
  static final ServiceFactory _instance = ServiceFactory._internal();

  factory ServiceFactory() {
    return _instance;
  }

  // Constructor privado
  ServiceFactory._internal() {
    _initializeServices();
  }
}
```

Listing 2: Uso de Singleton en ServiceFactory

7.2. 2. Abstract Factory

Este patrón centraliza la creación de familias de objetos. Lo apliqué en la clase `ServiceFactory`. Esta “súper fábrica” es la única que sabe cómo construir los servicios concretos de la aplicación. Así, la interfaz gráfica es agnóstica de la implementación.

```
/// Obtiene la instancia Singleton del servicio de autenticacion
IAuthService getAuthService() => _authService;

/// Obtiene la instancia Singleton del servicio de superheroes
ISuperheroService getSuperheroService() => _superheroService;
```

Listing 3: Abstract Factory produciendo servicios

7.3. 3. Factory Method

Este patrón lo apliqué en mis modelos para delegar la lógica pesada de cómo se construyen a partir de los JSON que manda la API. Creé un archivo `superhero_factory.dart` para centralizar esto, pero también lo usé internamente en los modelos.

Esto me ayudó muchísimo cuando la API mandaba los datos anidados de forma rara, permitiéndome tener extracciones seguras (resilientes) sin llenar la vista de condicionales.

```

factory Comment.fromJson(Map<String, dynamic> json) {
  String parsedUsername = 'Usuario';

  // Extraccion profunda y segura del usuario
  if (json['user'] != null && json['user'] is Map) {
    parsedUsername = json['user']['username']?.toString() ?? 'Usuario';
  } else if (json['username'] != null) {
    parsedUsername = json['username'].toString();
  }
  // ...
}

```

Listing 4: Factory Method extrayendo el username anidado

7.4. 4. Builder

El Builder lo implementé en `query_builder.dart`. Lo utilicé porque al hacer búsquedas o filtros de superhéroes, a veces solo quieres buscar por nombre, a veces por habilidad, y tener un constructor con 20 parámetros opcionales es sucio. El Builder me permitió armar la consulta de la API paso a pasito.

8. Patrones Estructurales

Estos patrones me ayudaron a armar el rompecabezas de clases, logrando que trabajaran juntas pero sin estar fuertemente acopladas.

8.1. 5. Facade (Fachada)

Este patrón fue la salvación para la interfaz gráfica. Lo ocupé creando la clase `AppCoordinator` (`app_coordinator.dart`).

Antes, mi pantalla de inicio (`home_screen`) tenía que llamar al servicio de héroes, al de comentarios y al de reacciones al mismo tiempo. Era un caos. Con el Facade, la UI solo le habla al `AppCoordinator`, y este hace el trabajo sucio por detrás.

```

class AppCoordinator {
  // ...
  /// Crea un comentario coordinando la llamada
  Future<Comment> createComment(int superheroId, String text) async {
    return _commentService.createComment(superheroId, text);
  }

  /// Obtiene reacciones disponibles
  Future<List<Reaction>> getAvailableReactions() async {
    return _reactionService.fetchAvailableReactions();
  }
}

```

Listing 5: Facade ocultando la complejidad de llamadas múltiples

8.2. 6. Adapter (Adaptador)

Este patrón fue clave cuando me enfrenté a los errores de compatibilidad con Google Chrome y Dart. La API me mandaba listas en diferentes formatos, lo que hacía que el código colapsara con el terrible error `JSArray<dynamic>`.

Para arreglarlo, creé la clase `ListResponseAdapter`. Básicamente “adapta” lo que sea que mande la API y lo convierte a la fuerza en una lista limpia y tipada.

```
class ListResponseAdapter {
  static List<T> adaptList<T>(  
    dynamic response, {  
    required T Function(Map<String, dynamic>) parser,  
  }) {  
    if (response is List) {  
      return response.map((item) => parser(item as Map<String, dynamic>))  
    }.toList();  
    }  
    // Intenta adaptarse si la lista viene dentro de 'content'  
    if (response is Map<String, dynamic> && response['content'] is List)  
    {  
      return (response['content'] as List).map((item) => parser(item)).  
    toList();  
    }  
    throw Exception('Formato de respuesta no soportado');  
  }  
}
```

Listing 6: ListResponseAdapter salvando la compatibilidad web

8.3. 7. Decorator (Decorador)

El Decorador me permitió agregarle comportamiento extra a mis servicios sin tener que modificar ni romper su código original. De hecho, me resultó tan útil que lo implementé de dos formas diferentes:

- **Logging Decorator** (`logging_decorator.dart`): En lugar de ensuciar mis servicios con `print()` para depurar, hice un envoltorio que imprime automáticamente el tiempo, el éxito o el error de cada petición a la API.
- **Cache Decorator** (`cache_decorator.dart`): Para no saturar la API pidiendo los mismos datos una y otra vez si el usuario recarga la pantalla, este decorador guarda temporalmente las respuestas y las reutiliza de forma inteligente.

```
abstract class LoggingDecorator<T> {  
  // ...  
  Future<R> executeWithLogging<R>(String methodName, Future<R> Function  
    () operation) async {  
    try {  
      logOperation(methodName); // Imprime inicio  
      final result = await operation(); // Ejecuta el servicio real  
      logSuccess(methodName, result: result); // Imprime exito  
      return result;  
    } catch (e, stackTrace) {  
      logError(methodName, e, stackTrace);  
      rethrow;  
    }  
  }  
}
```



```

    }
  }
}

```

Listing 7: Decorator envolviendo ejecuciones para hacer logs

8.4. 8. Proxy

Utilicé el patrón Proxy en el archivo `service_proxy.dart`. Actúa como un “cadenero” antes de dejar que la app consuma un servicio. De hecho, el archivo contiene tres variantes de este patrón para proteger la aplicación desde distintos frentes:

- **AuthenticationProxy:** Valida si el usuario tiene un Token válido en memoria antes de siquiera intentar hacer la petición HTTP.
- **LazyServiceProxy:** Retrasa la creación de ciertos servicios pesados en memoria hasta que el usuario realmente los necesita.
- **RateLimitingProxy:** Evita que un usuario haga demasiadas peticiones seguidas (spam de clics) y sature el servidor.

Gracias a estas tres barreras, evito sobrecargar la API con peticiones que sé de antemano que serán rechazadas o que son repetitivas.

9. Retos Técnicos Superados

A lo largo de este refactoring, me topé con dos errores severos que demostraron la importancia de una buena arquitectura:

1. **Bucle Infinito en la API:** Inicialmente, Flutter arrojaba un error de CORS (`Failed to fetch`). Al investigar, descubrí que la API en Spring Boot entraba en recursión al serializar las relaciones bidireccionales entre *Superheroe* y *Reacción*. Al implementar DTOs (Data Transfer Objects) y usar `@JsonIgnore` en el Backend, liberé la API y Flutter pudo recibir la respuesta sana.
2. **Casteos Estrictos (Dart Web):** Al ejecutar la app en Chrome, el motor detectaba que las listas venían como `dynamic` y detenía la ejecución en los componentes visuales. La solución no fue parchar la vista, sino blindar la capa de datos forzando tipados fuertes como `Future<List<Reaction>`, demostrando el valor del tipado estricto.

10. Beneficios Obtenidos y Visión a Futuro

Después de tantas horas trabajando con el código, moviendo archivos y arreglando errores de dependencias, mi proyecto pasó de ser un archivo frágil a un ecosistema robusto. Para demostrar la efectividad de esto, planteo dos escenarios hipotéticos:

- **¿Qué pasa si un día usamos Firebase para el login?**
Gracias al Principio DIP y al *Abstract Factory*, la interfaz gráfica ni se enteraría del cambio. Solo tendría que crear un `FirebaseAuthServiceImpl`, decirle a mi fábrica que lo use, y listo.

- **¿Qué pasa si el backend vuelve a cambiar el JSON?**

Si mañana el servidor manda los datos con otros nombres, no habrá pánico. Gracias al patrón *Adapter* y al *Factory Method*, solo tengo que actualizar una línea en los mapeos, manteniendo intactas todas las pantallas.

11. Conclusión

Para cerrar este reporte, quiero mencionar que al iniciar esta materia percibía los patrones de diseño como algo muy abstracto, simples diagramas. Sin embargo, al enfrentarme a problemas reales de integración en este proyecto, me di cuenta de que son salvavidas indispensables.

Refactorizar el FrontEnd de “Superheroes” fue un gran reto. Hubo momentos donde arreglaba una cosa y se rompían tres más, pero justo ahí entendí el valor de tener el código separado en capas y con responsabilidades únicas.

Al final, este proyecto me demostró que ser ingeniera de software no se trata solo de hacer que una aplicación se vea bonita por encima, sino de construir bases matemáticas y lógicas sólidas por debajo. Hoy puedo decir que entrego un sistema modular, leíble y listo para crecer, cumpliendo con los estándares de calidad de la industria.